Natural Language Processing

Practical Lab Manual

Academic Year: 2026-27

Name: Aditya Shirsatrao

GitHub: adityashirsatrao007

12 Experiments covering:

Tokenization, WordNet, N-grams, Stemming,

HMM, NER, SRL, Classification, Sentiment,

RNN, LSTM, and Word Sense Disambiguation

Experiment 01

Tokenization & Word Frequency

Source Code

```
"""
Expt 1: Convert text into tokens. Find word frequency.
"""
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.probability import FreqDist
import matplotlib.pyplot as plt

nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)

text = """
Natural language processing (NLP) is a subfield of linguistics, computer science,
and artificial intelligence concerned with the interactions between computers and
human language. The goal is to enable computers to understand, interpret, and
generate human language in a valuable way. NLP combines computational linguistics
with statistical, machine learning, and deep learning models.
"""

print("=" * 60)
print("EXPERIMENT 1: Tokenization & Word Frequency")
print("=" * 60)

# Sentence tokenization
sentences = sent_tokenize(text)
print(f"\nSentence Tokenization ({len(sentences)} sentences):")
for i, s in enumerate(sentences, 1):
    print(f"  [{i}] {s.strip()}")

# Word tokenization
words = word_tokenize(text.lower())
print(f"\nWord Tokenization ({len(words)} tokens):")
print(f"  {words}")

# Word frequency
freq = FreqDist(words)
print(f"\nTop 15 Word Frequencies:")
for word, count in freq.most_common(15):
    print(f"  '{word}': {count}")

# Plot
plt.figure(figsize=(10, 5))
freq.plot(15, title="Word Frequency Distribution", marker="o")
plt.tight_layout()
plt.savefig("outputs/ex01_word_frequency.png", dpi=150)
plt.close()
print("\nPlot saved to outputs/ex01_word_frequency.png")
```

Experiment 01 - Output

Output

```
============================================================
EXPERIMENT 1: Tokenization & Word Frequency
============================================================

Sentence Tokenization (3 sentences):
  [1] Natural language processing (NLP) is a subfield of linguistics, computer science,
and artificial intelligence concerned with the interactions between computers and
human language.
  [2] The goal is to enable computers to understand, interpret, and
generate human language in a valuable way.
  [3] NLP combines computational linguistics
with statistical, machine learning, and deep learning models.

Word Tokenization (63 tokens):
  ['natural', 'language', 'processing', '(', 'nlp', ')', 'is', 'a', 'subfield', 'of', 'linguistics', ',', 'com

Top 15 Word Frequencies:
  ',': 6
  'and': 4
  'language': 3
  '.': 3
  'nlp': 2
  'is': 2
  'a': 2
  'linguistics': 2
  'with': 2
  'the': 2
  'computers': 2
  'human': 2
  'to': 2
  'learning': 2
  'natural': 1

Plot saved to outputs/ex01_word_frequency.png
```

Experiment 02

Synonyms & Antonyms using WordNet

Source Code

```
"""
Expt 2: Find synonym / antonym of a word using WordNet.
"""
import nltk
from nltk.corpus import wordnet as wn

nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)

print("=" * 60)
print("EXPERIMENT 2: Synonyms & Antonyms using WordNet")
print("=" * 60)

words = ["good", "happy", "fast", "big", "smart"]

for word in words:
    synonyms = set()
    antonyms = set()

    for syn in wn.synsets(word):
        for lemma in syn.lemmas():
            synonyms.add(lemma.name())
            if lemma.antonyms():
                antonyms.add(lemma.antonyms()[0].name())

    print(f"\nWord: '{word}'")
    print(f"  Synonyms : {sorted(synonyms)[:10]}")
    print(f"  Antonyms : {sorted(antonyms)[:10]}")

    # WordNet hierarchy
    synsets = wn.synsets(word)
    if synsets:
        s = synsets[0]
        print(f"  Definition: {s.definition()}")
        print(f"  Examples  : {s.examples()}")
        hypernyms = s.hypernyms()
        if hypernyms:
            print(f"  Hypernym  : {hypernyms[0].name()}")
```

Experiment 02 - Output

Output

```
============================================================
EXPERIMENT 2: Synonyms & Antonyms using WordNet
============================================================

Word: 'good'
  Synonyms : ['adept', 'beneficial', 'commodity', 'dear', 'dependable', 'effective', 'estimable', 'expert', 'f
  Antonyms : ['bad', 'badness', 'evil', 'evilness', 'ill']
  Definition: benefit
  Examples  : ['for your own good', "what's the good of worrying?"]
  Hypernym  : advantage.n.01

Word: 'happy'
  Synonyms : ['felicitous', 'glad', 'happy', 'well-chosen']
  Antonyms : ['unhappy']
  Definition: enjoying or showing or marked by joy or pleasure
  Examples  : ['a happy smile', 'spent many happy days on the beach', 'a happy marriage']

Word: 'fast'
  Synonyms : ['debauched', 'degenerate', 'degraded', 'dissipated', 'dissolute', 'fast', 'fasting', 'firm', 'fl
  Antonyms : ['slow']
  Definition: abstaining from food
  Examples  : []
  Hypernym  : abstinence.n.02

Word: 'big'
  Synonyms : ['adult', 'bad', 'big', 'bighearted', 'boastful', 'boastfully', 'bounteous', 'bountiful', 'bragga
  Antonyms : ['little', 'small']
  Definition: above average in size or number or quantity or magnitude or extent
  Examples  : ['a large city', 'set out for the big city', 'a large sum', 'a big (or large) barn', 'a large fa

Word: 'smart'
  Synonyms : ['ache', 'bright', 'chic', 'fresh', 'hurt', 'impertinent', 'impudent', 'overbold', 'sassy', 'sauc
  Antonyms : ['stupid']
  Definition: a kind of pain such as that caused by a wound or a burn or a sore
  Examples  : []
  Hypernym  : pain.n.01
```

Experiment 03

Bigram/Trigram Language Model & Regex

Source Code

```
"""
Expt 3: Bigram / Trigram Language Model. Generate regex for given text.
"""
import nltk
from nltk import bigrams, trigrams, FreqDist
from nltk.probability import ConditionalFreqDist
import re

nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)

text = """
The cat sat on the mat. The dog sat on the log. The cat chased the dog.
The dog chased the cat. The mat was on the floor. The log was on the grass.
"""

print("=" * 60)
print("EXPERIMENT 3: Bigram/Trigram Language Model & Regex")
print("=" * 60)

tokens = nltk.word_tokenize(text.lower())
tokens = [t for t in tokens if t.isalpha()]

# Bigrams
print("\n--- Bigrams ---")
bg = list(bigrams(tokens))
print(f"Total bigrams: {len(bg)}")
cfd_bigram = ConditionalFreqDist(bg)
print("Top bigrams:")
bg_freq = FreqDist(bg)
for (w1, w2), c in bg_freq.most_common(10):
    print(f"  ('{w1}', '{w2}'): {c}")

# Trigrams
print("\n--- Trigrams ---")
tg = list(trigrams(tokens))
print(f"Total trigrams: {len(tg)}")
tg_freq = FreqDist(tg)
print("Top trigrams:")
for (w1, w2, w3), c in tg_freq.most_common(10):
    print(f"  ('{w1}', '{w2}', '{w3}'): {c}")

# Conditional frequency: given a word, what follows?
print("\nConditional FreqDist (word -> next word):")
for word in ["the", "cat", "dog"]:
    if word in cfd_bigram:
        top = cfd_bigram[word].most_common(5)
        print(f"  After '{word}': {top}")

# Regex generation
print("\n--- Regex Patterns ---")
patterns = {
    "Email": r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}",
    "Phone (Indian)": r"\+?91[\s-]?\d{10}|\d{5}[\s-]\d{5}",
    "Date (DD/MM/YYYY)": r"\d{2}/\d{2}/\d{4}",
    "URL": r"https?://(?:www\.)?\S+",
    "IP Address": r"\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}",
}

samples = {
    "Email": "Contact us at support@example.com or admin@company.org",
    "Phone (Indian)": "Call +919876543210 or 11234567890",
    "Date (DD/MM/YYYY)": "Event on 25/12/2026 and 01/01/2027",
    "URL": "Visit https://www.example.com or http://test.org/page",
    "IP Address": "Server at 192.168.1.1 and 10.0.0.255",
}

for name, pattern in patterns.items():
    matches = re.findall(pattern, samples[name])
    print(f"  {name}: {matches}")
```

Experiment 03 - Output

Output

```
============================================================
EXPERIMENT 3: Bigram/Trigram Language Model & Regex
============================================================

--- Bigrams ---
Total bigrams: 33
Top bigrams:
  ('on', 'the'): 4
  ('the', 'cat'): 3
  ('the', 'dog'): 3
  ('sat', 'on'): 2
  ('the', 'mat'): 2
  ('the', 'log'): 2
  ('chased', 'the'): 2
  ('was', 'on'): 2
  ('cat', 'sat'): 1
  ('mat', 'the'): 1

--- Trigrams ---
Total trigrams: 32
Top trigrams:
  ('sat', 'on', 'the'): 2
  ('was', 'on', 'the'): 2
  ('the', 'cat', 'sat'): 1
  ('cat', 'sat', 'on'): 1
  ('on', 'the', 'mat'): 1
  ('the', 'mat', 'the'): 1
  ('mat', 'the', 'dog'): 1
  ('the', 'dog', 'sat'): 1
  ('dog', 'sat', 'on'): 1
  ('on', 'the', 'log'): 1

Conditional FreqDist (word -> next word):
  After 'the': [('cat', 3), ('dog', 3), ('mat', 2), ('log', 2), ('floor', 1)]
  After 'cat': [('sat', 1), ('chased', 1), ('the', 1)]
  After 'dog': [('sat', 1), ('the', 1), ('chased', 1)]

--- Regex Patterns ---
  Email: ['support@example.com', 'admin@company.org']
  Phone (Indian): ['+919876543210']
  Date (DD/MM/YYYY): ['25/12/2026', '01/01/2027']
  URL: ['https://www.example.com', 'http://test.org/page']
  IP Address: ['192.168.1.1', '10.0.0.255']
```

Experiment 04

Lemmatization, Stemming & POS Tagging

Source Code

```
"""
Expt 4: Lemmatization and Stemming. POS Tagging using Penn Treebank tag set.
"""
import nltk
from nltk.stem import PorterStemmer, LancasterStemmer, WordNetLemmatizer
from nltk.corpus import stopwords

nltk.download("averaged_perceptron_tagger", quiet=True)
nltk.download("averaged_perceptron_tagger_eng", quiet=True)
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)
nltk.download("stopwords", quiet=True)

print("=" * 60)
print("EXPERIMENT 4: Lemmatization, Stemming & POS Tagging")
print("=" * 60)

words = ["running", "flies", "studies", "better", "geese", "happiness",
         "cats", "swimming", "organized", "connection"]

# Stemming
porter = PorterStemmer()
lancaster = LancasterStemmer()
lemmatizer = WordNetLemmatizer()

print(f"\n{'Word':<15} {'Porter':<15} {'Lancaster':<15} {'Lemmatizer':<15}")
print("-" * 60)
for w in words:
    print(f"{w:<15} {porter.stem(w):<15} {lancaster.stem(w):<15} {lemmatizer.lemmatize(w):<15}")

# POS-based lemmatization
print("\n--- Lemmatization with POS tags ---")
pos_tests = [
    ("running", "v"), ("flies", "n"), ("better", "a"),
    ("geese", "n"), ("running", "n"), ("studies", "n"),
]
for word, pos in pos_tests:
    result = lemmatizer.lemmatize(word, pos=pos)
    print(f"  '{word}' (POS={pos}) -> '{result}'")

# POS Tagging with Penn Treebank tag set
print("\n--- POS Tagging (Penn Treebank) ---")
sentence = "The quick brown fox jumps over the lazy dog near the river bank"
tokens = nltk.word_tokenize(sentence)
tagged = nltk.pos_tag(tokens)
print(f"  Sentence: {sentence}")
print(f"  Tagged: {tagged}")

# Tag descriptions
print("\n--- Penn Treebank Tag Set ---")
tag_descriptions = {
    "CC": "Coordinating conjunction", "CD": "Cardinal number",
    "DT": "Determiner", "EX": "Existential there",
    "FW": "Foreign word", "IN": "Preposition/subordinating conjunction",
    "JJ": "Adjective", "JJR": "Adjective, comparative",
    "JJS": "Adjective, superlative", "MD": "Modal",
    "NN": "Noun, singular", "NNS": "Noun, plural",
    "NNP": "Proper noun, singular", "RB": "Adverb",
    "RBR": "Adverb, comparative", "TO": "to",
    "VB": "Verb, base form", "VBD": "Verb, past tense",
    "VBG": "Verb, gerund", "VBN": "Verb, past participle",
    "VBP": "Verb, non-3rd person singular present",
    "VBZ": "Verb, 3rd person singular present",
    "WDT": "Wh-determiner", "WP": "Wh-pronoun",
}
for word, tag in tagged:
    desc = tag_descriptions.get(tag, "Other")
    print(f"  {word:<12} -> {tag:<5} ({desc})")
```

Experiment 04 - Output

Output

```
============================================================
EXPERIMENT 4: Lemmatization, Stemming & POS Tagging
============================================================

Word            Porter          Lancaster       Lemmatizer
------------------------------------------------------------
running         run             run             running
flies           fli             fli             fly
studies         studi           study           study
better          better          bet             better
geese           gees            gees            goose
happiness       happi           happy           happiness
cats            cat             cat             cat
swimming        swim            swim            swimming
organized       organ           org             organized
connection      connect         connect         connection

--- Lemmatization with POS tags ---
  'running' (POS=v) -> 'run'
  'flies' (POS=n) -> 'fly'
  'better' (POS=a) -> 'good'
  'geese' (POS=n) -> 'goose'
  'running' (POS=n) -> 'running'
  'studies' (POS=n) -> 'study'

--- POS Tagging (Penn Treebank) ---
  Sentence: The quick brown fox jumps over the lazy dog near the river bank
  Tagged: [('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN'), ('jumps', 'VBZ'), ('over', 'IN'), (

--- Penn Treebank Tag Set ---
  The           -> DT    (Determiner)
  quick         -> JJ    (Adjective)
  brown         -> NN    (Noun, singular)
  fox           -> NN    (Noun, singular)
  jumps         -> VBZ   (Verb, 3rd person singular present)
  over          -> IN    (Preposition/subordinating conjunction)
  the           -> DT    (Determiner)
  lazy          -> JJ    (Adjective)
  dog           -> NN    (Noun, singular)
  near          -> IN    (Preposition/subordinating conjunction)
  the           -> DT    (Determiner)
  river         -> NN    (Noun, singular)
  bank          -> NN    (Noun, singular)
```

Experiment 05

HMM POS Tagger & Chunker

Source Code

```
"""
Expt 5: Implement HMM for POS tagging. Build a Chunker.
"""
import nltk
from nltk import RegexpParser
import random

nltk.download("brown", quiet=True)
nltk.download("nps_chat", quiet=True)
nltk.download("averaged_perceptron_tagger", quiet=True)
nltk.download("averaged_perceptron_tagger_eng", quiet=True)
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)

print("=" * 60)
print("EXPERIMENT 5: HMM POS Tagger & Chunker")
print("=" * 60)

# -- HMM-style POS Tagger using Bigram approach --
from nltk.corpus import brown

brown_tagged = brown.tagged_sents(categories="news", tagset="universal")

# Split train/test
split = int(len(brown_tagged) * 0.8)
train_sents = brown_tagged[:split]
test_sents = brown_tagged[split:]

# HMM transition probabilities (bigram tag model)
class HMMTagger:
    def __init__(self):
        self.tag_freq = {}
        self.transition = {}
        self.emission = {}

    def train(self, tagged_sents):
        for sent in tagged_sents:
            prev_tag = "<START>"
            for word, tag in sent:
                # Transition counts
                self.transition.setdefault(prev_tag, {})
                self.transition[prev_tag][tag] = self.transition[prev_tag].get(tag, 0) + 1
                # Emission counts
                self.emission.setdefault(tag, {})
                self.emission[tag][word] = self.emission[tag].get(word, 0) + 1
                # Tag counts
                self.tag_freq[tag] = self.tag_freq.get(tag, 0) + 1
                prev_tag = tag
            self.transition.setdefault(prev_tag, {})
            self.transition[prev_tag]["<END>"] = self.transition[prev_tag].get("<END>", 0) + 1

    def _transition_prob(self, prev_tag, tag):
        if prev_tag not in self.transition:
            return 1e-6
        total = sum(self.transition[prev_tag].values())
        return self.transition[prev_tag].get(tag, 0) / total

    def _emission_prob(self, tag, word):
        if tag not in self.emission:
            return 1e-6
        total = sum(self.emission[tag].values())
        return self.emission[tag].get(word, 1e-6) / total

    def _viterbi(self, words):
        tags = list(self.tag_freq.keys())
        n = len(words)
        if n == 0:
            return []

        # Viterbi table
        viterbi = [{}]
        backptr = [{}]

        for tag in tags:
            viterbi[0][tag] = self._transition_prob("<START>", tag) * self._emission_prob(tag, words[0])
    # ... (truncated for page limit)
    # See full code in GitHub repo
```

Experiment 05 - Output

Output

```
============================================================
EXPERIMENT 5: HMM POS Tagger & Chunker
============================================================

--- Training HMM POS Tagger (Brown corpus, news) ---
HMM Tagger Accuracy: 88.81%

Custom: 'The quick brown fox jumps over the lazy dog'
Tagged: [('The', 'DET'), ('quick', 'ADJ'), ('brown', 'NOUN'), ('fox', 'X'), ('jumps', 'X'), ('over', 'ADP'), (

--- NP Chunker (RegexpParser) ---
Chunks found:
  NP: The quick brown fox
  VP: jumps
  PP: over the lazy dog
  NP: the lazy dog

Tree saved to outputs/ex05_chunk_tree.txt
```

Experiment 06

Named Entity Recognition

Source Code

```
"""
Expt 6: Implement Named Entity Recognizer.
"""
import nltk
from nltk import ne_chunk, pos_tag, word_tokenize
from collections import Counter

nltk.download("maxent_ne_chunker", quiet=True)
nltk.download("maxent_ne_chunker_tab", quiet=True)
nltk.download("averaged_perceptron_tagger", quiet=True)
nltk.download("averaged_perceptron_tagger_eng", quiet=True)
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
nltk.download("words", quiet=True)

print("=" * 60)
print("EXPERIMENT 6: Named Entity Recognition (NER)")
print("=" * 60)

sentences = [
    "Barack Obama was born in Hawaii and served as the 44th President of the United States.",
    "Apple Inc. was founded by Steve Jobs in Cupertino, California.",
    "Google was created by Larry Page and Sergey Brin at Stanford University.",
    "Elon Musk founded SpaceX in Hawthorne and Tesla in Austin, Texas.",
    "Satya Nadella is the CEO of Microsoft in Redmond, Washington.",
]

entity_types = Counter()

for i, sent in enumerate(sentences, 1):
    print(f"\n[{i}] {sent}")
    tokens = word_tokenize(sent)
    tagged = pos_tag(tokens)
    tree = ne_chunk(tagged)

    entities = []
    for subtree in tree:
        if hasattr(subtree, "label"):
            entity_name = " ".join(word for word, tag in subtree.leaves())
            entity_type = subtree.label()
            entities.append((entity_name, entity_type))
            entity_types[entity_type] += 1

    if entities:
        print(f"  Entities found:")
        for name, etype in entities:
            print(f"    {etype}: {name}")
    else:
        print("  No entities found.")

print(f"\n--- Entity Type Distribution ---")
for etype, count in entity_types.most_common():
    print(f"  {etype}: {count}")

# Extract structured data
print("\n--- Structured Extraction ---")
for i, sent in enumerate(sentences, 1):
    tokens = word_tokenize(sent)
    tagged = pos_tag(tokens)
    tree = ne_chunk(tagged)

    people = []
    organizations = []
    locations = []

    for subtree in tree:
        if hasattr(subtree, "label"):
            name = " ".join(w for w, t in subtree.leaves())
            if subtree.label() == "PERSON":
                people.append(name)
            elif subtree.label() == "ORGANIZATION":
                organizations.append(name)
            elif subtree.label() == "GPE":
                locations.append(name)

    # ... (truncated for page limit)
    # See full code in GitHub repo
```

Experiment 06 - Output

Output

```
============================================================
EXPERIMENT 6: Named Entity Recognition (NER)
============================================================

[1] Barack Obama was born in Hawaii and served as the 44th President of the United States.
  Entities found:
    PERSON: Barack
    PERSON: Obama
    GPE: Hawaii
    GPE: United States

[2] Apple Inc. was founded by Steve Jobs in Cupertino, California.
  Entities found:
    PERSON: Apple
    ORGANIZATION: Inc.
    PERSON: Steve Jobs
    GPE: Cupertino
    GPE: California

[3] Google was created by Larry Page and Sergey Brin at Stanford University.
  Entities found:
    PERSON: Google
    PERSON: Larry Page
    PERSON: Sergey Brin
    ORGANIZATION: Stanford University

[4] Elon Musk founded SpaceX in Hawthorne and Tesla in Austin, Texas.
  Entities found:
    PERSON: Elon
    PERSON: Musk
    ORGANIZATION: SpaceX
    GPE: Hawthorne
    GPE: Tesla
    GPE: Austin
    GPE: Texas

[5] Satya Nadella is the CEO of Microsoft in Redmond, Washington.
  Entities found:
    PERSON: Satya
    ORGANIZATION: Nadella
    ORGANIZATION: CEO of Microsoft
    GPE: Redmond
    GPE: Washington

--- Entity Type Distribution ---
  PERSON: 10
  GPE: 10
  ORGANIZATION: 5

--- Structured Extraction ---

  Sentence 1:
    People      : ['Barack', 'Obama']
    Organizations: None
    Locations    : ['Hawaii', 'United States']

  Sentence 2:
    People      : ['Apple', 'Steve Jobs']
    Organizations: ['Inc.']
    Locations    : ['Cupertino', 'California']

  Sentence 3:
    People      : ['Google', 'Larry Page', 'Sergey Brin']
    Organizations: ['Stanford University']
    Locations    : None

  Sentence 4:
    People      : ['Elon', 'Musk']
    Organizations: ['SpaceX']
    Locations    : ['Hawthorne', 'Tesla', 'Austin', 'Texas']

  Sentence 5:
    People      : ['Satya']
    Organizations: ['Nadella', 'CEO of Microsoft']
    Locations    : ['Redmond', 'Washington']
```

Experiment 07

Semantic Role Labelling

Source Code

```
"""
Expt 7: Implement Semantic Role Labelling to identify named entities.
"""
import nltk
import re

nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
nltk.download("averaged_perceptron_tagger", quiet=True)
nltk.download("averaged_perceptron_tagger_eng", quiet=True)

print("=" * 60)
print("EXPERIMENT 7: Semantic Role Labelling (SRL)")
print("=" * 60)

# Simplified SRL using pattern-based approach
# In production, useAllenNLP or other SRL models

def extract_srl(sentence):
    """Extract basic semantic roles using pattern matching and dependency heuristics."""
    tokens = nltk.word_tokenize(sentence)
    tagged = nltk.pos_tag(tokens)

    roles = {
        "AGENT": [],       # Who did it (subject)
        "ACTION": [],      # What happened (verb)
        "PATIENT": [],     # Who/what was affected (object)
        "INSTRUMENT": [],  # How/with what
        "LOCATION": [],    # Where
        "TIME": [],        # When
    }

    # Find verb (ACTION)
    verb_idx = None
    for i, (word, tag) in enumerate(tagged):
        if tag.startswith("VB") and tag != "VBG":
            roles["ACTION"].append(word)
            verb_idx = i
            break

    if verb_idx is None:
        return roles

    # Agent: NPs before verb (NNP, NN, PRP, DT+NN)
    for i in range(verb_idx - 1, -1, -1):
        word, tag = tagged[i]
        if tag in ("NNP", "NNPS", "PRP", "NN", "NNS"):
            roles["AGENT"].insert(0, word)
        elif tag == "DT":
            roles["AGENT"].insert(0, word)
            break
        elif tag in ("IN", "TO", "WDT", "WP"):
            break

    # Patient: NPs after verb
    for i in range(verb_idx + 1, len(tagged)):
        word, tag = tagged[i]
        if tag in ("NNP", "NNPS", "NN", "NNS", "PRP", "DT"):
            roles["PATIENT"].append(word)
        elif tag == "IN":
            # Check for location/time
            if i + 1 < len(tagged):
                next_word, next_tag = tagged[i + 1]
                if next_tag in ("NNP", "NNPS"):
                    roles["LOCATION"].append(next_word)
            break

    # Location markers
    loc_preps = {"in", "at", "on", "near", "from", "to", "into"}
    for i, (word, tag) in enumerate(tagged):
        if word.lower() in loc_preps and i + 1 < len(tagged):
            next_word, next_tag = tagged[i + 1]
            if next_tag in ("NNP", "NNPS"):
                if not roles["LOCATION"]:
                    roles["LOCATION"].append(next_word)
    # ... (truncated for page limit)
    # See full code in GitHub repo
```

Experiment 07 - Output

Output

```
============================================================
EXPERIMENT 7: Semantic Role Labelling (SRL)
============================================================

[1] Barack Obama signed the healthcare bill in Washington on Monday.
  AGENT       : Barack Obama
  ACTION      : signed
  PATIENT     : the healthcare bill
  LOCATION    : Washington

[2] The chef cooked a delicious meal in the kitchen.
  AGENT       : The chef
  ACTION      : cooked
  PATIENT     : a meal

[3] Google acquired YouTube for 1.65 billion dollars.
  AGENT       : Google
  ACTION      : acquired
  PATIENT     : YouTube
  TIME        : 1.65 billion

[4] Mary sent a letter to John from New York.
  AGENT       : Mary
  ACTION      : sent
  PATIENT     : a letter John
  LOCATION    : New

[5] The teacher explained the concept to students in the classroom.
  AGENT       : The teacher
  ACTION      : explained
  PATIENT     : the concept students

--- Combined NER + SRL Analysis ---

[1] Barack Obama signed the healthcare bill in Washington on Monday.
  POS Tags: [('Barack', 'NNP'), ('Obama', 'NNP'), ('signed', 'VBD'), ('Washington', 'NNP'), ('Monday', 'NNP')]
  Semantic Roles:
    AGENT        = Barack Obama
    ACTION       = signed
    PATIENT      = the healthcare bill
    LOCATION     = Washington

[2] The chef cooked a delicious meal in the kitchen.
  POS Tags: [('cooked', 'VBD')]
  Semantic Roles:
    AGENT        = The chef
    ACTION       = cooked
    PATIENT      = a meal

[3] Google acquired YouTube for 1.65 billion dollars.
  POS Tags: [('Google', 'NNP'), ('acquired', 'VBD'), ('YouTube', 'NNP')]
  Semantic Roles:
    AGENT        = Google
    ACTION       = acquired
    PATIENT      = YouTube
    TIME         = 1.65 billion

[4] Mary sent a letter to John from New York.
  POS Tags: [('Mary', 'NNP'), ('sent', 'VBD'), ('John', 'NNP'), ('New', 'NNP'), ('York', 'NNP')]
  Semantic Roles:
    AGENT        = Mary
    ACTION       = sent
    PATIENT      = a letter John
    LOCATION     = New

[5] The teacher explained the concept to students in the classroom.
  POS Tags: [('explained', 'VBD')]
  Semantic Roles:
    AGENT        = The teacher
    ACTION       = explained
    PATIENT      = the concept students
```

Experiment 08

Text Classifier (Logistic Regression)

Source Code

```
"""
Expt 8: Implement text classifier using logistic regression model.
"""
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import numpy as np

print("=" * 60)
print("EXPERIMENT 8: Text Classifier (Logistic Regression)")
print("=" * 60)

# Dataset: news topic classification
texts = [
    # Sports
    "The team won the championship game in overtime",
    "The player scored a hat trick in the football match",
    "The Olympics will be held in Paris next year",
    "The tennis player won the Grand Slam title",
    "The basketball team drafted a new star player",
    "The cricket match ended in a draw on the final day",
    "The soccer coach was fired after losing streak",
    "The swimmer broke the world record in the 100m freestyle",
    "The boxing match was declared a knockout victory",
    "The rugby team scored a try in the last minute",
    # Technology
    "The new iPhone has an improved camera and processor",
    "Artificial intelligence is transforming healthcare industry",
    "The startup raised 50 million dollars in funding",
    "Google released a new update for Android operating system",
    "The cloud computing market is growing rapidly",
    "Machine learning models are being used for fraud detection",
    "The cryptocurrency market crashed overnight",
    "Quantum computing breakthrough achieved by researchers",
    "The robot uses deep learning for navigation",
    "Blockchain technology is revolutionizing supply chain",
    # Politics
    "The president signed the new trade agreement today",
    "The senator proposed a new healthcare reform bill",
    "Elections will be held in November across the country",
    "The government announced new environmental policies",
    "The prime minister addressed the parliament on budget",
    "New legislation aims to regulate social media platforms",
    "The diplomat met with foreign minister for peace talks",
    "The opposition party criticized the ruling government",
    "Voter turnout was highest in the last decade",
    "The congress passed the infrastructure spending bill",
    # Science
    "Scientists discovered a new species in the Amazon rainforest",
    "The Mars rover found evidence of ancient water",
    "Researchers developed a new vaccine for tropical disease",
    "The telescope captured images of distant galaxies",
    "Climate change is causing glaciers to melt faster",
    "The experiment confirmed the theory of quantum entanglement",
    "Biologists mapped the complete genome of a new organism",
    "The particle accelerator detected a new subatomic particle",
    "Astronomers found an exoplanet in the habitable zone",
    "The clinical trial showed promising results for cancer treatment",
]

labels = (
    ["Sports"] * 10 +
    ["Technology"] * 10 +
    ["Politics"] * 10 +
    ["Science"] * 10
)

# TF-IDF Vectorization
vectorizer = TfidfVectorizer(stop_words="english", max_features=500)
X = vectorizer.fit_transform(texts)
y = np.array(labels)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
    # ... (truncated for page limit)
    # See full code in GitHub repo
```

Experiment 08 - Output

Output

```
============================================================
EXPERIMENT 8: Text Classifier (Logistic Regression)
============================================================

--- Model Performance ---
              precision    recall  f1-score   support

    Politics       0.67      1.00      0.80         2
     Science       0.17      0.50      0.25         2
      Sports       1.00      0.50      0.67         2
  Technology       0.00      0.00      0.00         4

    accuracy                           0.40        10
   macro avg       0.46      0.50      0.43        10
weighted avg       0.37      0.40      0.34        10

Confusion Matrix:
[[2 0 0 0]
 [1 1 0 0]
 [0 1 1 0]
 [0 4 0 0]]

--- Predictions on New Texts ---
  'The quarterback threw a touchdown pass' -> Science
  'Apple announced the new MacBook Pro' -> Politics
  'The election results were announced today' -> Politics
  'Researchers found a cure for the disease' -> Science
  'The stock market hit record highs' -> Sports

--- Top Features per Class ---
  Politics: ['november', 'elections', 'country', 'new', 'infrastructure']
  Science: ['zone', 'exoplanet', 'astronomers', 'habitable', 'theory']
  Sports: ['match', 'won', 'scored', 'team', 'player']
  Technology: ['computing', 'improved', 'iphone', 'processor', 'camera']
```

Experiment 09

Movie Reviews Sentiment Classifier

Source Code

```
"""
Expt 9: Implement a movie reviews sentiment classifier.
"""
import nltk
from nltk.corpus import movie_reviews, stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
import random

nltk.download("movie_reviews", quiet=True)
nltk.download("punkt", quiet=True)
nltk.download("punkt_tab", quiet=True)
nltk.download("stopwords", quiet=True)

print("=" * 60)
print("EXPERIMENT 9: Movie Reviews Sentiment Classifier")
print("=" * 60)

# Load movie reviews dataset
documents = []
labels = []

for category in movie_reviews.categories():
    for fileid in movie_reviews.fileids(category):
        doc = " ".join(movie_reviews.words(fileid))
        documents.append(doc)
        labels.append(1 if category == "pos" else 0)

print(f"Dataset: {len(documents)} reviews ({labels.count(1)} positive, {labels.count(0)} negative)")

# Shuffle
combined = list(zip(documents, labels))
random.shuffle(combined)
documents, labels = zip(*combined)

# TF-IDF
stop_words = set(stopwords.words("english"))
vectorizer = TfidfVectorizer(
    max_features=5000,
    stop_words="english",
    ngram_range=(1, 2),
    min_df=2,
    max_df=0.95
)
X = vectorizer.fit_transform(documents)
y = list(labels)

# Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train Logistic Regression
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

# Evaluate
y_pred = model.predict(X_test)
print(f"\n--- Model Performance ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}")
print(classification_report(y_test, y_pred, target_names=["Negative", "Positive"]))

# Predict custom reviews
print("--- Custom Review Predictions ---")
custom_reviews = [
    "This movie was absolutely fantastic! Great acting and storyline.",
    "Terrible film. Waste of time and money. The plot was boring.",
    "An okay movie. Not great but not terrible either.",
    "One of the best movies I have ever seen. Highly recommended!",
    "The acting was wooden and the dialogue was cringe-worthy.",
    "A masterpiece of cinema with brilliant performances.",
    "I fell asleep halfway through. Very dull and predictable.",
    "Stunning visuals and a gripping narrative from start to finish.",
]
    # ... (truncated for page limit)
    # See full code in GitHub repo
```

Experiment 09 - Output

Output

```
============================================================
EXPERIMENT 9: Movie Reviews Sentiment Classifier
============================================================
Dataset: 2000 reviews (1000 positive, 1000 negative)

--- Model Performance ---
Accuracy: 83.50%
              precision    recall  f1-score   support

    Negative       0.85      0.82      0.83       201
    Positive       0.82      0.85      0.84       199

    accuracy                           0.83       400
   macro avg       0.84      0.84      0.83       400
weighted avg       0.84      0.83      0.83       400

--- Custom Review Predictions ---

  Review: 'This movie was absolutely fantastic! Great acting and storyl...'
  Sentiment: POSITIVE (confidence: 61.1%)

  Review: 'Terrible film. Waste of time and money. The plot was boring....'
  Sentiment: NEGATIVE (confidence: 89.5%)

  Review: 'An okay movie. Not great but not terrible either....'
  Sentiment: NEGATIVE (confidence: 53.9%)

  Review: 'One of the best movies I have ever seen. Highly recommended!...'
  Sentiment: POSITIVE (confidence: 80.2%)

  Review: 'The acting was wooden and the dialogue was cringe-worthy....'
  Sentiment: NEGATIVE (confidence: 59.5%)

  Review: 'A masterpiece of cinema with brilliant performances....'
  Sentiment: POSITIVE (confidence: 76.9%)

  Review: 'I fell asleep halfway through. Very dull and predictable....'
  Sentiment: NEGATIVE (confidence: 64.4%)

  Review: 'Stunning visuals and a gripping narrative from start to fini...'
  Sentiment: POSITIVE (confidence: 61.2%)

--- Top Features ---
  0: ['life', 'great', 'truman', 'war', 'mulan', 'excellent', 'world', 'perfect', 'american', 'best']
```

Experiment 10

RNN for Sequence Labelling

Source Code

```
"""
Expt 10: Implement RNN for sequence labelling.
"""
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense, Embedding, TimeDistributed
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

print("=" * 60)
print("EXPERIMENT 10: RNN for Sequence Labelling (POS Tagging)")
print("=" * 60)

# Simple POS tagging dataset
training_data = [
    (["The", "cat", "sat", "on", "the", "mat"],
     ["DT", "NN", "VBD", "IN", "DT", "NN"]),
    (["I", "love", "natural", "language", "processing"],
     ["PRP", "VBP", "JJ", "NN", "NN"]),
    (["She", "runs", "every", "morning"],
     ["PRP", "VBZ", "DT", "NN"]),
    (["The", "dog", "chased", "the", "cat", "quickly"],
     ["DT", "NN", "VBD", "DT", "NN", "RB"]),
    (["We", "are", "learning", "deep", "learning"],
     ["PRP", "VBP", "VBG", "JJ", "NN"]),
    (["He", "will", "arrive", "tomorrow"],
     ["PRP", "MD", "VB", "NN"]),
    (["The", "big", "brown", "fox", "jumped"],
     ["DT", "JJ", "JJ", "NN", "VBD"]),
    (["She", "gave", "him", "a", "book"],
     ["PRP", "VBD", "PRP", "DT", "NN"]),
    (["They", "played", "soccer", "yesterday"],
     ["PRP", "VBD", "NN", "NN"]),
    (["The", "student", "studied", "hard", "for", "the", "exam"],
     ["DT", "NN", "VBD", "RB", "IN", "DT", "NN"]),
    (["I", "ate", "a", "delicious", "meal"],
     ["PRP", "VBD", "DT", "JJ", "NN"]),
    (["The", "car", "is", "fast"],
     ["DT", "NN", "VBZ", "JJ"]),
    (["Birds", "fly", "in", "the", "sky"],
     ["NNS", "VBP", "IN", "DT", "NN"]),
    (["She", "reads", "books", "every", "day"],
     ["PRP", "VBZ", "NNS", "DT", "NN"]),
    (["The", "teacher", "explained", "the", "concept"],
     ["DT", "NN", "VBD", "DT", "NN"]),
]

# Build vocabularies
word_tags = set()
all_words = set()
all_tags = set()
for words, tags in training_data:
    for w, t in zip(words, tags):
        all_words.add(w)
        all_tags.add(t)
        word_tags.add((w, t))

word2idx = {w: i + 2 for i, w in enumerate(sorted(all_words))}
word2idx["<PAD>"] = 0
word2idx["<UNK>"] = 1
tag2idx = {t: i for i, t in enumerate(sorted(all_tags))}
idx2tag = {i: t for t, i in tag2idx.items()}

VOCAB_SIZE = len(word2idx)
NUM_TAGS = len(tag2idx)
MAX_LEN = max(len(words) for words, _ in training_data)

print(f"Vocabulary: {VOCAB_SIZE} words")
print(f"Tags: {NUM_TAGS} ({list(sorted(all_tags))})")
print(f"Max sequence length: {MAX_LEN}")

# Prepare data
X = []
y = []
    # ... (truncated for page limit)
    # See full code in GitHub repo
```

Experiment 10 - Output

Output

```
  warnings.warn(
============================================================
EXPERIMENT 10: RNN for Sequence Labelling (POS Tagging)
============================================================
Vocabulary: 61 words
Tags: 13 (['DT', 'IN', 'JJ', 'MD', 'NN', 'NNS', 'PRP', 'RB', 'VB', 'VBD', 'VBG', 'VBP', 'VBZ'])
Max sequence length: 7
X shape: (15, 7)
y shape: (15, 7, 13)
Model: "sequential"
+---------------------------------?-----------------------?---------------+
| Layer (type)                    | Output Shape          |       Param # |
?---------------------------------?-----------------------?---------------?
| embedding (Embedding)           | ?                     |   0 (unbuilt) |
+---------------------------------+-----------------------+---------------+
| simple_rnn (SimpleRNN)          | ?                     |   0 (unbuilt) |
+---------------------------------+-----------------------+---------------+
| time_distributed                | ?                     |   0 (unbuilt) |
| (TimeDistributed)               |                       |               |
+---------------------------------+-----------------------+---------------+
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
--- Training ---
Final accuracy: 100.00%
Final loss: 0.0481
--- Predictions ---
  Words: ['The', 'cat', 'sat', 'on', 'the', 'mat']
  Predicted POS: ['DT', 'NN', 'VBD', 'IN', 'DT', 'NN']
  True POS:      ['DT', 'NN', 'VBD', 'IN', 'DT', 'NN']
  Words: ['She', 'runs', 'every', 'morning']
  Predicted POS: ['PRP', 'VBZ', 'DT', 'NN']
  True POS:      ['DT', 'NN', 'VBD', 'IN']
  Words: ['I', 'love', 'natural', 'language']
  Predicted POS: ['PRP', 'VBP', 'JJ', 'NN']
  True POS:      ['DT', 'NN', 'VBD', 'IN']
```

Experiment 11

POS Tagging using LSTM

Source Code

```
"""
Expt 11: POS tagging using LSTM.
"""
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Embedding, LSTM, Dense, TimeDistributed, Bidirectional
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
from sklearn.model_selection import train_test_split

print("=" * 60)
print("EXPERIMENT 11: POS Tagging using LSTM")
print("=" * 60)

# Penn Treebank-like dataset
sentences_data = [
    (["The", "cat", "sat", "on", "the", "mat"],
     ["DT", "NN", "VBD", "IN", "DT", "NN"]),
    (["I", "love", "natural", "language", "processing"],
     ["PRP", "VBP", "JJ", "NN", "NN"]),
    (["She", "runs", "every", "morning"],
     ["PRP", "VBZ", "DT", "NN"]),
    (["The", "dog", "chased", "the", "cat", "quickly"],
     ["DT", "NN", "VBD", "DT", "NN", "RB"]),
    (["We", "are", "learning", "deep", "learning"],
     ["PRP", "VBP", "VBG", "JJ", "NN"]),
    (["He", "will", "arrive", "tomorrow"],
     ["PRP", "MD", "VB", "NN"]),
    (["The", "big", "brown", "fox", "jumped", "over", "the", "lazy", "dog"],
     ["DT", "JJ", "JJ", "NN", "VBD", "IN", "DT", "JJ", "NN"]),
    (["She", "gave", "him", "a", "beautiful", "book"],
     ["PRP", "VBD", "PRP", "DT", "JJ", "NN"]),
    (["They", "played", "soccer", "yesterday"],
     ["PRP", "VBD", "NN", "NN"]),
    (["The", "student", "studied", "hard", "for", "the", "exam"],
     ["DT", "NN", "VBD", "RB", "IN", "DT", "NN"]),
    (["I", "ate", "a", "delicious", "meal"],
     ["PRP", "VBD", "DT", "JJ", "NN"]),
    (["The", "car", "is", "very", "fast"],
     ["DT", "NN", "VBZ", "RB", "JJ"]),
    (["Birds", "fly", "high", "in", "the", "sky"],
     ["NNS", "VBP", "RB", "IN", "DT", "NN"]),
    (["She", "reads", "many", "books", "every", "day"],
     ["PRP", "VBZ", "JJ", "NNS", "DT", "NN"]),
    (["The", "teacher", "explained", "the", "concept", "clearly"],
     ["DT", "NN", "VBD", "DT", "NN", "RB"]),
    (["A", "small", "cat", "sleeps", "on", "the", "warm", "bed"],
     ["DT", "JJ", "NN", "VBZ", "IN", "DT", "JJ", "NN"]),
    (["The", "children", "played", "in", "the", "park"],
     ["DT", "NNS", "VBD", "IN", "DT", "NN"]),
    (["My", "friend", "works", "at", "a", "technology", "company"],
     ["PRP$", "NN", "VBZ", "IN", "DT", "NN", "NN"]),
    (["The", "old", "man", "walked", "slowly"],
     ["DT", "JJ", "NN", "VBD", "RB"]),
    (["We", "should", "protect", "the", "environment"],
     ["PRP", "MD", "VB", "DT", "NN"]),
]

# Augment data by generating variations
augmented = []
for words, tags in sentences_data:
    augmented.append((words, tags))
    # Slight variations
    if len(words) > 3:
        augmented.append((words[:len(words)//2], tags[:len(tags)//2]))
        augmented.append((words[len(words)//2:], tags[len(tags)//2:]))

sentences_data = augmented

# Build vocabularies
all_words = set()
all_tags = set()
for words, tags in sentences_data:
    for w, t in zip(words, tags):
    # ... (truncated for page limit)
    # See full code in GitHub repo
```

Experiment 11 - Output

Output

```
============================================================
EXPERIMENT 11: POS Tagging using LSTM
============================================================
Vocabulary: 88 words
Tags: 14 (['DT', 'IN', 'JJ', 'MD', 'NN', 'NNS', 'PRP', 'PRP$', 'RB', 'VB', 'VBD', 'VBG', 'VBP', 'VBZ'])
Max length: 9
Training samples: 48, Validation: 12
Model: "functional"
+---------------------?------------------?------------?-------------------+
| Layer (type)        | Output Shape     |    Param # | Connected to      |
?---------------------?------------------?------------?-------------------?
| input_layer         | (None, 9)        |          0 | -                 |
| (InputLayer)        |                  |            |                   |
+---------------------+------------------+------------+-------------------+
| embedding           | (None, 9, 64)    |      5,632 | input_layer[0][0] |
| (Embedding)         |                  |            |                   |
+---------------------+------------------+------------+-------------------+
| not_equal           | (None, 9)        |          0 | input_layer[0][0] |
| (NotEqual)          |                  |            |                   |
+---------------------+------------------+------------+-------------------+
| bidirectional       | (None, 9, 256)   |    197,632 | embedding[0][0],  |
| (Bidirectional)     |                  |            | not_equal[0][0]   |
+---------------------+------------------+------------+-------------------+
| time_distributed    | (None, 9, 15)    |      3,855 | bidirectional[0]? |
| (TimeDistributed)   |                  |            | not_equal[0][0]   |
+---------------------+------------------+------------+-------------------+
 Total params: 207,119 (809.06 KB)
 Trainable params: 207,119 (809.06 KB)
 Non-trainable params: 0 (0.00 B)
--- Training BiLSTM ---
Training accuracy:   100.00%
Validation accuracy: 75.86%
--- Predictions ---
  Words: ['The', 'cat', 'sat', 'on', 'the', 'mat']
  LSTM POS: ['DT', 'NN', 'VBD', 'IN', 'DT', 'NN']
  Words: ['She', 'runs', 'every', 'morning']
  LSTM POS: ['PRP', 'VBZ', 'DT', 'NN']
  Words: ['I', 'love', 'deep', 'learning']
  LSTM POS: ['VBP', 'VBP', 'JJ', 'NN']
  Words: ['The', 'big', 'brown', 'fox', 'jumped']
  LSTM POS: ['DT', 'JJ', 'JJ', 'NN', 'VBD']
```

Experiment 12

Word Sense Disambiguation (LSTM/GRU)

Source Code

```
"""
Expt 12: Word Sense Disambiguation by LSTM/GRU.
"""
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (
    Input, Embedding, LSTM, GRU, Dense,
    Bidirectional, Dropout, concatenate, GlobalMaxPooling1D
)
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical

print("=" * 60)
print("EXPERIMENT 12: Word Sense Disambiguation (LSTM/GRU)")
print("=" * 60)

# WSD dataset: ambiguous words with different senses
# "bank" -> financial institution vs river bank
# "light" -> not heavy vs illumination
# "bat" -> animal vs sports equipment
# "crane" -> bird vs machine
# "spring" -> season vs water source vs jump

wsd_data = [
    # bank (sense 0: financial, sense 1: river)
    ("I deposited money at the bank", "bank", 0),
    ("The bank approved my loan application", "bank", 0),
    ("She works at a bank downtown", "bank", 0),
    ("The bank charges high interest rates", "bank", 0),
    ("He invested his savings in the bank", "bank", 0),
    ("The river bank was eroded by floods", "bank", 1),
    ("We sat on the bank of the river", "bank", 1),
    ("Fish were swimming near the bank", "bank", 1),
    ("The bank of the river was muddy", "bank", 1),
    ("She walked along the river bank", "bank", 1),
    # light (sense 0: not heavy, sense 1: illumination)
    ("The bag is very light to carry", "light", 0),
    ("She wore a light dress in summer", "light", 0),
    ("This laptop is surprisingly light", "light", 0),
    ("The feather is light as air", "light", 0),
    ("He picked up the light box easily", "light", 0),
    ("The light from the candle was dim", "light", 1),
    ("Turn on the light in the room", "light", 1),
    ("The sunlight streaming through the window", "light", 1),
    ("She carried a torch light at night", "light", 1),
    ("The light was too bright to look at", "light", 1),
    # bat (sense 0: animal, sense 1: sports equipment)
    ("The bat flew out of the cave at dusk", "bat", 0),
    ("Bats use echolocation to navigate", "bat", 0),
    ("A bat hung upside down from the ceiling", "bat", 0),
    ("The vampire bat is found in tropical regions", "bat", 0),
    ("He hit the ball with the bat", "bat", 1),
    ("The cricket bat is made of willow", "bat", 1),
    ("She swung the bat and scored a six", "bat", 1),
    ("The baseball bat cracked on impact", "bat", 1),
    ("He brought his bat to the game", "bat", 1),
    # crane (sense 0: bird, sense 1: machine)
    ("The crane spread its wings and flew away", "crane", 0),
    ("A beautiful crane stood by the lake", "crane", 0),
    ("The crane is an endangered species", "crane", 0),
    ("Cranes migrate south in winter", "crane", 0),
    ("The construction crane lifted the beam", "crane", 1),
    ("They used a crane to build the bridge", "crane", 1),
    ("The crane operator worked from dawn to dusk", "crane", 1),
    ("A giant crane towered over the construction site", "crane", 1),
]

# Build vocabulary
all_words = set()
for sent, _, _ in wsd_data:
    for w in sent.split():
        all_words.add(w.lower())

word2idx = {w: i + 2 for i, w in enumerate(sorted(all_words))}
    # ... (truncated for page limit)
    # See full code in GitHub repo
```

Experiment 12 - Output

Output

```
  warnings.warn(
  warnings.warn(
============================================================
EXPERIMENT 12: Word Sense Disambiguation (LSTM/GRU)
============================================================
Dataset: 37 examples, 136 vocabulary
Senses: 2 (0=first meaning, 1=second meaning)
=== Model 1: Bidirectional LSTM ===
LSTM Test Accuracy: 62.50%
=== Model 2: Bidirectional GRU ===
GRU Test Accuracy: 50.00%
--- Predictions on New Sentences ---
  'I need to go to the bank to withdraw cash'
  Target word: 'bank'
  LSTM: financial (conf: 100.0%)
  GRU:  river (conf: 100.0%)
  'The bank of the river was full of rocks'
  Target word: 'bank'
  LSTM: river (conf: 100.0%)
  GRU:  river (conf: 100.0%)
  'Please turn on the light'
  Target word: 'light'
  LSTM: illumination (conf: 99.3%)
  GRU:  illumination (conf: 100.0%)
  'This package is very light'
  Target word: 'light'
  LSTM: lightweight (conf: 100.0%)
  GRU:  lightweight (conf: 100.0%)
  'The crane flew over the wetland'
  Target word: 'crane'
  LSTM: machine (conf: 100.0%)
  GRU:  machine (conf: 99.8%)
  'The crane lifted the heavy steel beam'
  Target word: 'crane'
  LSTM: machine (conf: 100.0%)
  GRU:  machine (conf: 100.0%)
--- Comparison ---
  LSTM accuracy: 62.50%
  GRU  accuracy: 50.00%
```